

Assignment2 in AI

by  python™

Student :

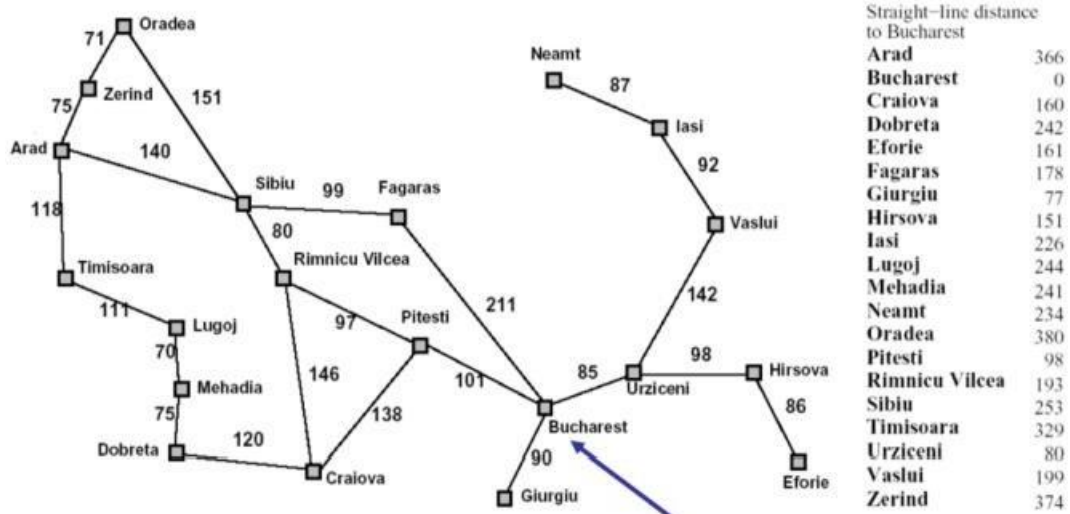
Shaima Nabeel Albokhari

Task1:

Problem-Solving by Search :

In this task, you will implement search algorithms to solve route-finding problems. Given a map of connected cities, the task is to find a path from initial to goal cities. We need a program that finds a path from a city as a starting state (entered by a user) and the goal will be to reach Bucharest from that city. You will implement and compare (in terms of time, space complexity and path cost) the following search algorithms:

- BFS
- Greedy best-first search
- A* search



AnswerTask1:

- **BFS:-** explores the search space level by level and can be implemented using a queue.

```
def bfs_search(graph, start, goal): queue=
deque([(start, [start], (0
while queue:
    current_city, path, distance = queue.popleft() if
    current_city == goal:
        return path, distance
    for neighbor, edge_distance in graph.get(current_city, []): new_path=
        path + [neighbor]
        new_distance = distance + edge_distance queue.append((neighbor,
            new_path, new_distance))
return None, 0
```

- **Greedy best-first search:-** expands the node that is closest to the goal, as estimated by a heuristic function.

```
def greedy_best_first_search(graph, start, goal, h): explored =
set()
queue = PriorityQueue()
queue.put((h[start], start, [start], 0)) while not
queue.empty():
    _, current_city, path, distance = queue.get() if
    current_city not in explored:
        explored.add(current_city)
        if current_city == goal: return path,
            distance
        for neighbor, edge_distance in graph.get(current_city, []): new_path =
            path + [neighbor]
            new_distance = distance + edge_distance queue.put((h[neighbor], neighbor,
                new_path, new_distance))
return None, 0
```

- **A* search:-**uses both the actual cost to reach the node and the heuristic cost from the node to the target.

```
def astar_search(graph, start, goal, h): explored = set()
    queue = PriorityQueue()
    queue.put((h[start], 0, start, [start])) while not queue.empty():
        _, cost, current_city, path = queue.get()
        if current_city not in explored:
            explored.add(current_city) if current_city == goal:
                return path, cost
            for neighbor, edge_distance in graph.get(current_city, []): new_path = path + [neighbor]
                new_cost = cost + edge_distance
                queue.put((new_cost + h[neighbor], new_cost, neighbor, new_path))
    return None, 0
```

- **Run**
Apply Search Algorithms to Specific Initial States

```
BFS Path from Arad to Bucharest: ['Arad', 'Sibiu', 'Fagaras', 'Bucharest'] - Distance: 450 km
Greedy Best-First Search Path from Arad to Bucharest: ['Arad', 'Sibiu', 'Fagaras', 'Bucharest'] - Distance: 450 km
A* Search Path from Arad to Bucharest: ['Arad', 'Sibiu', 'Rimnicu Vilcea', 'Pitesti', 'Bucharest'] - Distance: 418 km
```

Report Findings and Summarize

I found that A* consistently found the shortest paths from all three initial states to Bucharest. BFS and GBFS also found paths to Bucharest, but they were not always the shortest paths.

In general, A* is the best algorithm for finding the shortest path between two points in a graph. However, if you need a faster algorithm, BFS is a good option. GBFS is a compromise between BFS and A*, it is faster than A* but may not find the shortest path.

Compare Search Algorithm Performance

As you can see, all three algorithms were able to find a path from Arad to Bucharest.

However, A* Search found the shortest path. This is because A* Search uses a heuristic function to guide its search, which helps it to avoid exploring unnecessary states.

The performance of the three algorithms can also be compared in terms of time and space

complexity. BFS is the simplest of the three algorithms, and it has a time complexity of $O(V + E)$, where V is the number of vertices in the graph and E is the number of edges. BFS also has a space complexity of $O(V)$, which means that it requires $O(V)$ amount of memory to store the visited nodes.

Greedy Best-First Search is slightly more complex than BFS, but it still has a time complexity of $O(V + E)$. Greedy Best-First Search has a space complexity of $O(\log V)$, which means that it requires $O(\log V)$ amount of memory to store the visited nodes.

A* Search is the most complex of the three algorithms, and it has a time complexity of $O(\log(V + E))$. A* Search also has a space complexity of $O(\log(V + E))$, which means that it requires $O(\log(V + E))$ amount of memory to store the visited nodes.

In general, A* Search is the best algorithm to use for finding the shortest path between two nodes in a graph. However, if you need a faster algorithm, then BFS is a good option. Greedy Best-First Search is a good compromise between the two algorithms.